

Python for Big Data Engineering

25 Core Concepts, Practiced by Example

This guide walks through the Python fundamentals that show up constantly in real data engineering work — the same building blocks behind pandas, T-SQL scripting, and PySpark pipelines. Each of the 25 chapters explains one concept, calls out why it matters specifically for data engineering, and works through a runnable example built around order/sales data. Every code example in this PDF is also collected, in the same order, into a single companion file: **script.py**, so you can run, tweak, and extend everything hands-on.

Section	Chapters	Covers
Language Foundations	Ch. 1–12	Types, data structures, control flow, functions, comprehensions, map/filter/reduce
Data I/O	Ch. 13–15	Reading and writing text, CSV, and JSON — the raw materials of any pipeline
Robust, Reusable Code	Ch. 16–20	Exceptions, modules, generators, context managers, classes
Practical Data Tools	Ch. 21–24	Dates, regex, collections, and filesystem paths
Capstone	Ch. 25	A complete mini ETL pipeline tying every concept together

Table of Contents

01	Variables & Data Types
02	Numbers & Arithmetic Operators
03	Strings & String Methods
04	Lists
05	Tuples & Sets
06	Dictionaries
07	Conditionals (if / elif / else)
08	Loops: for, while, break, continue
09	Functions & Arguments (*args, **kwargs)
10	Lambda Functions
11	Comprehensions (list, dict, set)
12	Functional Tools: map, filter, reduce
13	File I/O (Reading & Writing Text Files)
14	Working with CSV
15	Working with JSON
16	Exception Handling (try / except / finally)
17	Modules & Packages
18	Iterators & Generators
19	Context Managers (the `with` statement)
20	Classes & OOP Basics
21	Working with Dates & Times (datetime)
22	Regular Expressions for Data Cleaning
23	The collections Module
24	Working with Files & Paths (os, pathlib)
25	Capstone: A Mini ETL Pipeline

01 Variables & Data Types

A variable is a name bound to a value in memory. Python is dynamically typed, so the same name can point to an int, a float, a string, a boolean, or None at different times, and Python figures the type out at runtime with `type()`.

WHY IT MATTERS FOR DATA ENGINEERING

Every column in a Spark DataFrame or SQL table ultimately maps back to one of these core Python types when you inspect a row. Knowing exactly which type you are holding avoids silent bugs, like adding a string "5" to an int 5.

example_01.py — run this from script.py

```
# ---- Core data types ----
record_id = 1042          # int
price = 19.99            # float
product = "USB-C Cable"  # str
in_stock = True          # bool
discount = None          # NoneType (missing value)

print(type(record_id), type(price), type(product), type(in_stock), type(discount))

# Type conversion (casting) - very common when reading raw text data
raw_value = "150"
quantity = int(raw_value)      # str -> int
unit_price = float("4.50")     # str -> float
label = str(quantity)          # int -> str

print(quantity * unit_price, "=>", label)
```

Practice task: Given the string '2026-07-25', split it into year, month, day and cast each part to int.

02 Numbers & Arithmetic Operators

Python supports the usual arithmetic operators (+ - * /), plus integer division //, modulo % for remainders, and ** for exponents. Comparison operators return booleans that drive filtering logic.

WHY IT MATTERS FOR DATA ENGINEERING

Modulo and integer division are the backbone of partitioning logic in distributed systems — deciding which of N worker nodes or which of N output files a record belongs to is almost always `hash(key) % num_partitions`.

example_02.py — run this from script.py

```
# ---- Arithmetic ----
total_rows = 1_045_000
num_partitions = 8

rows_per_partition = total_rows // num_partitions # integer division
leftover_rows = total_rows % num_partitions        # remainder

print(f"{num_partitions} partitions of {rows_per_partition} rows, "
      f"{leftover_rows} leftover")

# Simulating a hash-based partition assignment (the core idea behind
# how Spark/Hadoop decide which node a record goes to)
def assign_partition(key, num_partitions):
    return hash(key) % num_partitions

for customer_id in ["C-101", "C-102", "C-103", "C-104"]:
    print(customer_id, "->", "partition", assign_partition(customer_id, num_partitions))
```

Practice task: Write a function that returns True if a row number should go into an even-numbered or odd-numbered output file, using %.

03 Strings & String Methods

Strings are immutable sequences of characters. The methods `.strip()` `.split()` `.join()` `.lower()` `.replace()` cover the vast majority of real-world text cleaning, and f-strings (`f"..."`) are the modern way to format output.

WHY IT MATTERS FOR DATA ENGINEERING

Raw data almost never arrives clean: extra whitespace, inconsistent casing, stray punctuation, and mixed delimiters are the norm in log files, CSV exports, and scraped text — string methods are the first line of defense before any of it reaches a schema.

example_03.py — run this from script.py

```
# ---- String cleaning pipeline ----
raw = "  Product-Name:  USB-C Cable ,  Category : Electronics  \n"

cleaned = raw.strip()           # remove leading/trailing whitespace
parts = cleaned.split(",")      # split into fields
fields = [p.strip() for p in parts] # strip each field

result = {}
for field in fields:
    key, value = field.split(":")
    result[key.strip().lower()] = value.strip()

print(result)

# f-strings for building messages / file paths
year, month = 2026, 7
path = f"data/sales/year={year}/month={month:02d}/part-0001.csv"
print(path)

# join is the inverse of split - useful for rebuilding delimited rows
row_values = ["1042", "USB-C Cable", "19.99"]
print(",".join(row_values))
```

Practice task: Take the string `'name=cable|price=19.99|qty=10'` and turn it into a dictionary of the three key/value pairs.

04 Lists

A list is an ordered, mutable collection. Lists are the default structure for holding a batch of records in memory: rows read from a file, results of a query, or a column of values.

WHY IT MATTERS FOR DATA ENGINEERING

Any time you page a query result set into memory, or hold a batch before writing a file, you're holding a list. Slicing (`data[0:1000]`) is exactly how you'd chunk a large in-memory dataset into batches.

example_04.py — run this from script.py

```
# --- Lists as row batches ---
sales = [120.50, 89.99, 305.00, 12.75, 450.00, 60.00]

sales.append(99.99)          # add a new record
sales.remove(12.75)          # remove a specific value
total = sum(sales)
average = total / len(sales)

print("Total:", round(total, 2), "| Average:", round(average, 2))

# Slicing - chunking a list into batches (like a mini-batch reader)
batch_size = 3
for i in range(0, len(sales), batch_size):
    batch = sales[i:i + batch_size]
    print("Batch:", batch)

# Sorting
top_sales = sorted(sales, reverse=True)[:3]
print("Top 3 sales:", top_sales)
```

Practice task: Given a list of 10 order totals, split it into batches of 4 and print each batch's sum.

05 Tuples & Sets

A tuple is an ordered, **immutable** collection — ideal for a fixed record like a coordinate or a (key, value) pair. A set is an unordered collection of unique values, ideal for de-duplication and membership tests.

WHY IT MATTERS FOR DATA ENGINEERING

MapReduce works over (key, value) tuples at every stage. Sets are the fastest way to de-duplicate IDs or check "have I already processed this record?" against millions of items, since lookups are $O(1)$.

example_05.py — run this from script.py

```
# ---- Tuples: fixed, immutable records ----
point = (10.5, 20.3)
key_value_pair = ("python", 1)      # exactly what a Map phase emits

x, y = point                          # unpacking
print(f"x={x}, y={y}")

# ---- Sets: uniqueness & fast membership checks ----
customer_ids_today = {"C-101", "C-102", "C-103"}
customer_ids_yesterday = {"C-102", "C-103", "C-104"}

new_customers = customer_ids_today - customer_ids_yesterday
returning_customers = customer_ids_today & customer_ids_yesterday

print("New today:", new_customers)
print("Returning:", returning_customers)

seen_order_ids = set()
orders = ["O-1", "O-2", "O-1", "O-3", "O-2"]
duplicates = [o for o in orders if o in seen_order_ids or seen_order_ids.add(o)]
print("Duplicate order IDs:", duplicates)
```

Practice task: Given two lists of user IDs from two different days, find users present on both days using sets.

06 Dictionaries

A dictionary stores key-value pairs with $O(1)$ average lookup by key. It is the natural in-memory representation of a JSON object, a single database row, or an aggregation result like a word count.

WHY IT MATTERS FOR DATA ENGINEERING

Every reduce step you write — grouping, counting, summing by key — ends up as a dictionary. It's also how you'll represent a single record before writing it out as a row of JSON or Parquet.

example_06.py — run this from script.py

```
# --- Dictionaries for aggregation ---
orders = [
    {"customer": "C-101", "amount": 120.50},
    {"customer": "C-102", "amount": 89.99},
    {"customer": "C-101", "amount": 45.00},
    {"customer": "C-103", "amount": 305.00},
]

totals_by_customer = {}
for order in orders:
    customer = order["customer"]
    amount = order["amount"]
    totals_by_customer[customer] = totals_by_customer.get(customer, 0) + amount

for customer, total in totals_by_customer.items():
    print(customer, "->", round(total, 2))

# .get() with a default avoids KeyError; .setdefault() is handy for grouping
groups = {}
for order in orders:
    groups.setdefault(order["customer"], []).append(order["amount"])
print(groups)
```

Practice task: Given a list of `{'product': ..., 'qty': ...}` dicts, build a dictionary mapping `product -> total quantity sold`.

07 Conditionals (if / elif / else)

Conditionals branch program flow based on boolean expressions. Chained if / elif / else blocks are how you implement validation rules and business logic row by row.

WHY IT MATTERS FOR DATA ENGINEERING

Data quality checks — "is this value null?", "is this date in a valid range?", "does this row match the expected schema?" — are conditionals applied to every record in a pipeline.

example_07.py — run this from script.py

```
# --- Row validation with conditionals ---
def validate_order(order):
    if order.get("amount") is None:
        return "REJECTED: missing amount"
    elif order["amount"] < 0:
        return "REJECTED: negative amount"
    elif order["amount"] == 0:
        return "WARNING: zero-amount order"
    else:
        return "OK"

test_orders = [
    {"amount": 120.50},
    {"amount": -5},
    {"amount": 0},
    {},
]

for order in test_orders:
    print(order, "->", validate_order(order))

# Ternary (inline) conditional - common for quick flags
amount = 250
tier = "premium" if amount >= 200 else "standard"
print(tier)
```

Practice task: Write a function that labels an order's amount as 'small' (<50), 'medium' (50-500), or 'large' (>500).

08 Loops: for, while, break, continue

for loops iterate over a known sequence; while loops repeat until a condition changes. break exits a loop early, continue skips to the next iteration.

WHY IT MATTERS FOR DATA ENGINEERING

Reading a file line by line, paginating an API, or polling a job's status until it finishes are all natural while-loops. Row-by-row processing before you learn vectorized pandas/Spark operations is the for-loop pattern underneath almost every framework.

example_08.py — run this from script.py

```
# ---- for loop over records, with continue/break ----
rows = [10, -2, 15, 0, 8, -1, 20]
valid_total = 0

for value in rows:
    if value < 0:
        continue          # skip invalid rows, keep processing
    if value == 0:
        break              # stop entirely once we hit a sentinel value
    valid_total += value

print("Valid total before sentinel:", valid_total)

# ---- while loop: paginate until no more data ----
def fetch_page(page_number):
    """Pretend API call - returns [] once we run out of pages."""
    fake_pages = {1: ["a", "b"], 2: ["c", "d"], 3: []}
    return fake_pages.get(page_number, [])

page = 1
all_records = []
while True:
    data = fetch_page(page)
    if not data:
        break
    all_records.extend(data)
    page += 1

print("Fetched:", all_records)
```

Practice task: Simulate reading a stream of numbers and stop as soon as the running total exceeds 100.

09 Functions & Arguments (*args, **kwargs)

Functions package reusable logic. Default arguments make parameters optional, `*args` collects extra positional arguments, and `**kwargs` collects extra keyword arguments — both are common in config-driven pipeline code.

WHY IT MATTERS FOR DATA ENGINEERING

Pipeline steps (extract, transform, load) are almost always written as functions with a config dictionary or keyword arguments so the same function can run against dev, staging, and production settings.

example_09.py — run this from script.py

```
# --- Functions as pipeline steps ---
def extract(source, limit=None):
    data = list(range(1, 21))
    return data[:limit] if limit else data

def transform(data, *funcs):
    """Apply any number of transformation functions in sequence."""
    for func in funcs:
        data = [func(x) for x in data]
    return data

def load(data, destination="warehouse", **options):
    print(f>Loading {len(data)} rows into {destination} with options {options}")

raw = extract("orders_table", limit=5)
clean = transform(raw, lambda x: x * 2, lambda x: x + 1)
load(clean, destination="warehouse", mode="append", compress=True)

print(raw, "->", clean)
```

Practice task: Write a function `apply_discount(price, *, rate=0.1)` that only accepts `rate` as a keyword argument, and returns the discounted price.

10 Lambda Functions

A lambda is a small, anonymous, single-expression function. It is syntactic sugar for a throwaway function you only need in one place, most often as an argument to `sorted()`, `map()`, `filter()`.

WHY IT MATTERS FOR DATA ENGINEERING

Sort keys, filter predicates, and custom aggregation logic passed into pandas `.apply()` or Spark's `.map()` are written as lambdas constantly — knowing the syntax cold saves real time.

example_10.py — run this from script.py

```
orders = [
    {"id": "0-1", "amount": 120.50},
    {"id": "0-2", "amount": 89.99},
    {"id": "0-3", "amount": 305.00},
]

# Sorting with a lambda as the key
by_amount = sorted(orders, key=lambda o: o["amount"], reverse=True)
print([o["id"] for o in by_amount])

# A lambda stored in a variable (equivalent to a small def)
square = lambda x: x * x
print(square(9))

# Lambdas are most useful passed directly into another function
tax_rate = 0.14
with_tax = list(map(lambda o: {"id": o["id"], "amount": round(o["amount"] * (1 + tax_rate), 2)}, orders))
print(with_tax)
```

Practice task: Sort a list of product dicts by 'price' ascending using `sorted()` and a lambda key.

11 Comprehensions (list, dict, set)

Comprehensions build a new list, dict, or set from an existing iterable in a single, readable expression, optionally filtered with an `if` clause.

WHY IT MATTERS FOR DATA ENGINEERING

Comprehensions are the Pythonic equivalent of a SQL `SELECT . . . WHERE`: transform-and-filter in one line. They are also noticeably faster than an equivalent for-loop with `.append()`.

example_11.py — run this from script.py

```
orders = [
    {"id": "0-1", "amount": 120.50, "status": "paid"},
    {"id": "0-2", "amount": 89.99, "status": "pending"},
    {"id": "0-3", "amount": 305.00, "status": "paid"},
]

# List comprehension: transform + filter
paid_amounts = [o["amount"] for o in orders if o["status"] == "paid"]
print(paid_amounts)

# Dict comprehension: build a lookup table
amount_by_id = {o["id"]: o["amount"] for o in orders}
print(amount_by_id)

# Set comprehension: unique statuses
statuses = {o["status"] for o in orders}
print(statuses)

# Nested comprehension: flatten a list of lists (like flattening batches)
batches = [[1, 2, 3], [4, 5], [6]]
flat = [value for batch in batches for value in batch]
print(flat)
```

Practice task: Given a list of numbers 1-30, build a list of their squares but only for numbers divisible by 3, using a single comprehension.

12 Functional Tools: map, filter, reduce

`map(func, iterable)` applies a function to every element, `filter(func, iterable)` keeps only elements where the function returns True, and `functools.reduce(func, iterable)` folds a sequence down to a single value.

WHY IT MATTERS FOR DATA ENGINEERING

These three functions are the literal namesakes of the Map and Reduce phases covered earlier: `map()` transforms each record independently, and `reduce()` combines many values into one — exactly what happens across a distributed cluster.

example_12.py — run this from script.py

```
from functools import reduce

prices = [19.99, 4.50, 250.00, 12.75]

# map: transform every element (the "Map" phase)
with_tax = list(map(lambda p: round(p * 1.14, 2), prices))
print("With tax:", with_tax)

# filter: keep only matching elements
expensive = list(filter(lambda p: p > 20, prices))
print("Expensive:", expensive)

# reduce: fold everything into a single value (the "Reduce" phase)
total = reduce(lambda acc, p: acc + p, prices, 0)
print("Total:", round(total, 2))

# The word-count Reduce step from earlier, written with reduce()
word_counts = {"python": [1, 1, 1], "spark": [1], "data": [1]}
totals = {word: reduce(lambda a, b: a + b, counts) for word, counts in word_counts.items()}
print(totals)
```

Practice task: Use `map()` and `filter()` together to get the square of every even number from 1 to 20.

13 File I/O (Reading & Writing Text Files)

The `open()` function, used with a `with` block, reads or writes files and automatically closes them even if an error occurs. Iterating a file object line by line never loads the whole file into memory at once.

WHY IT MATTERS FOR DATA ENGINEERING

Log files, raw CSV/JSON exports, and staged data before it lands in a warehouse are all plain files. Line-by-line reading is exactly how you'd process a file too large to fit in RAM.

example_13.py — run this from script.py

```
# ---- Writing ----
with open("orders_log.txt", "w") as f:
    f.write("0-1,120.50,paid\n")
    f.write("0-2,89.99,pending\n")
    f.write("0-3,305.00,paid\n")

# ---- Appending ----
with open("orders_log.txt", "a") as f:
    f.write("0-4,12.75,paid\n")

# ---- Reading line by line (memory-safe, even for huge files) ----
paid_total = 0.0
with open("orders_log.txt", "r") as f:
    for line in f:
        order_id, amount, status = line.strip().split(",")
        if status == "paid":
            paid_total += float(amount)

print("Total paid:", round(paid_total, 2))
```

Practice task: Write 5 lines of 'name,score' data to a file, then read it back and print only rows where score > 50.

14 Working with CSV

The built-in `csv` module reads and writes CSV files correctly — handling quoted fields and embedded commas — which manual `.split(",")` cannot do reliably. `csv.DictReader` yields each row as a dictionary.

WHY IT MATTERS FOR DATA ENGINEERING

CSV remains one of the most common interchange formats for data pipelines. Knowing the standard library approach matters even after you move on to pandas, because pandas' CSV parser follows the same rules.

example_14.py — run this from script.py

```
import csv

rows = [
    {"order_id": "0-1", "amount": "120.50", "status": "paid"},
    {"order_id": "0-2", "amount": "89.99", "status": "pending"},
    {"order_id": "0-3", "amount": "305.00", "status": "paid"},
]

# ---- Writing CSV ----
with open("orders.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["order_id", "amount", "status"])
    writer.writeheader()
    writer.writerows(rows)

# ---- Reading CSV as dictionaries ----
with open("orders.csv", "r", newline="") as f:
    reader = csv.DictReader(f)
    paid_orders = [row for row in reader if row["status"] == "paid"]

print(paid_orders)
print("Paid count:", len(paid_orders))
```

Practice task: Read `orders.csv` and write a new file `paid_orders.csv` containing only rows where `status == 'paid'`.

15 Working with JSON

The `json` module converts between Python objects and JSON text: `json.dumps()` serializes, `json.loads()` parses, and `json.dump()/load()` work directly with file objects.

WHY IT MATTERS FOR DATA ENGINEERING

JSON is the default format for REST APIs, event streams (Kafka messages), and semi-structured data lakes. Nested JSON is also the shape you most often need to flatten before loading into a relational warehouse.

example_15.py — run this from script.py

```
import json

order = {
    "order_id": "0-1",
    "amount": 120.50,
    "status": "paid",
    "items": [
        {"sku": "USB-C", "qty": 2},
        {"sku": "HDMI", "qty": 1},
    ],
}

# ---- Serialize to a JSON string / file ----
json_text = json.dumps(order, indent=2)
print(json_text)

with open("order.json", "w") as f:
    json.dump(order, f, indent=2)

# ---- Parse JSON back into Python objects ----
with open("order.json", "r") as f:
    loaded = json.load(f)

# Flattening nested JSON into flat rows - a common data-engineering task
flat_rows = [
    {"order_id": loaded["order_id"], "sku": item["sku"], "qty": item["qty"]}
    for item in loaded["items"]
]
print(flat_rows)
```

Practice task: Given a JSON string representing a list of 3 users each with a nested 'address' dict, flatten each user into a single-level dict.

16 Exception Handling (try / except / finally)

try/except catches errors so one bad record does not crash an entire batch job. finally runs cleanup code regardless of success or failure, and you can raise your own exceptions with raise.

WHY IT MATTERS FOR DATA ENGINEERING

Production pipelines must be resilient: a single malformed row, a timed-out network call, or a missing file should be logged and skipped, not bring the whole job down. This is the difference between a script and a pipeline.

example_16.py — run this from script.py

```
raw_rows = ["100,20", "abc,30", "150,not_a_number", "80,15"]

def parse_row(row):
    price_str, qty_str = row.split(",")
    price = float(price_str)
    qty = int(qty_str)
    return price * qty

results = []
errors = []

for row in raw_rows:
    try:
        results.append(parse_row(row))
    except ValueError as e:
        errors.append((row, str(e)))
    finally:
        pass # e.g. always increment a "rows processed" counter here

print("Succeeded:", results)
print("Failed rows:", errors)

# Raising a custom, descriptive error
def load_config(config):
    if "warehouse" not in config:
        raise KeyError("config is missing required key 'warehouse'")
    return config

try:
    load_config({"mode": "append"})
except KeyError as e:
    print("Config error:", e)
```

Practice task: Process a list of strings meant to be integers; collect successfully-parsed ints in one list and the bad strings in another, without crashing.

17 Modules & Packages

A module is just a `.py` file; a package is a folder of modules. `import` brings in code from the standard library, third-party packages (pandas, pyspark), or your own files, keeping large projects organized.

WHY IT MATTERS FOR DATA ENGINEERING

Real pipelines are split across files — `extract.py`, `transform.py`, `load.py` — imported into a `main.py` that orchestrates them. This is the natural next step once a single script.py grows too large.

example_17.py — run this from script.py

```
# Standard library imports
import math
import datetime as dt
from collections import Counter

print(math.ceil(19.99))
print(dt.date.today())
print(Counter(["paid", "paid", "pending", "paid"]))

# Importing a specific function keeps the namespace clean
from statistics import mean, median
values = [10, 20, 30, 40]
print("mean:", mean(values), "| median:", median(values))

# If this were a project, you might structure it as:
#   pipeline/
#   __init__.py
#   extract.py  -> def extract(source): ...
#   transform.py -> def clean(df): ...
#   load.py     -> def load(df, target): ...
#   main.py     -> from pipeline import extract, transform, load
# and call: main.py orchestrates extract() -> transform() -> load()
```

Practice task: Use the `collections.Counter` class to count how many times each status ('paid','pending','paid','failed','paid') appears.

18 Iterators & Generators

A generator function uses `yield` instead of `return` to produce values one at a time, on demand, instead of building a whole list in memory. Generator expressions (`((x for x in ...))`) are the lazy counterpart to list comprehensions.

WHY IT MATTERS FOR DATA ENGINEERING

This is arguably the single most important Python concept for big data: it's how you process a file (or a billion-row stream) that does not fit in RAM — reading, transforming, and writing one record at a time instead of loading everything at once.

example_18.py — run this from script.py

```
# A generator that reads a huge file lazily, one line at a time
def read_large_file(path):
    with open(path, "r") as f:
        for line in f:
            yield line.strip()

# A generator that transforms values lazily (no intermediate list built)
def parse_amounts(lines):
    for line in lines:
        order_id, amount, status = line.split(",")
        yield {"order_id": order_id, "amount": float(amount), "status": status}

# Nothing runs yet - generators are lazy until you iterate them
lines = read_large_file("orders_log.txt")
parsed = parse_amounts(lines)

paid_total = sum(row["amount"] for row in parsed if row["status"] == "paid")
print("Paid total (streamed, low memory):", round(paid_total, 2))

# Generator expression: like a list comprehension, but lazy
squares = (n * n for n in range(1_000_000)) # instant - nothing computed yet
print(next(squares), next(squares), next(squares))
```

Practice task: Write a generator function that yields only even numbers from a range, then use it to sum the first 10 even numbers without building a full list.

19 Context Managers (the `with` statement)

A context manager guarantees setup and teardown code runs together, even if an exception happens in between — most commonly opening and reliably closing a resource like a file, database connection, or network socket via `with`.

WHY IT MATTERS FOR DATA ENGINEERING

Database and Spark session connections are expensive resources. Forgetting to close them leaks connections and can crash a long-running pipeline; `with` makes that impossible to forget.

example_19.py — run this from script.py

```
class DatabaseConnection:
    """A tiny stand-in for a real DB connection object."""
    def __enter__(self):
        print("Opening connection...")
        self.connected = True
        return self

    def __exit__(self, exc_type, exc_value, traceback):
        print("Closing connection...")
        self.connected = False
        return False # don't suppress exceptions

    def query(self, sql):
        return f"Executing: {sql}"

with DatabaseConnection() as conn:
    print(conn.query("SELECT * FROM orders"))
# connection is guaranteed closed here, even if query() had raised an error

# Multiple context managers at once - a common file-to-file pattern
with open("orders_log.txt") as infile, open("orders_copy.txt", "w") as outfile:
    for line in infile:
        outfile.write(line)
print("Copied file safely, both handles closed automatically.")
```

Practice task: Write a context manager (class or with `contextlib`) that prints 'START' and 'END' around a block of code.

20 Classes & OOP Basics

A class bundles data (attributes) and behavior (methods) into a reusable blueprint. `__init__` sets up a new instance's attributes, and methods operate on `self`.

WHY IT MATTERS FOR DATA ENGINEERING

Pipeline stages, connectors, and validators are often modeled as classes so the same `DataLoader` or `Validator` can be reused across many tables just by changing its configuration at construction time.

example_20.py — run this from script.py

```
class DataValidator:
    def __init__(self, required_fields):
        self.required_fields = required_fields
        self.errors = []

    def validate(self, row):
        missing = [f for f in self.required_fields if f not in row]
        if missing:
            self.errors.append((row, f"missing fields: {missing}"))
            return False
        return True

    def summary(self):
        return f"{len(self.errors)} row(s) failed validation"

class OrderValidator(DataValidator):
    """Inheritance: adds an amount-specific rule on top of the base class."""
    def validate(self, row):
        if not super().validate(row):
            return False
        if row.get("amount", 0) < 0:
            self.errors.append((row, "negative amount"))
            return False
        return True

rows = [
    {"order_id": "O-1", "amount": 120.50},
    {"order_id": "O-2", "amount": -5},
    {"amount": 40},
]

validator = OrderValidator(required_fields=["order_id", "amount"])
valid_rows = [r for r in rows if validator.validate(r)]

print("Valid rows:", valid_rows)
print(validator.summary())
```

Practice task: Create a class `Product` with attributes `name` and `price`, and a method `total_price(qty)` that returns `price * qty`.

21 Working with Dates & Times (datetime)

The `datetime` module represents dates and timestamps, supports arithmetic with `timedelta`, and converts to/from strings with `strftime()` (format) and `strptime()` (parse).

WHY IT MATTERS FOR DATA ENGINEERING

Nearly every pipeline is partitioned or scheduled by date (year=2026/month=07/day=25/), and log timestamps arrive as strings that must be parsed before they can be filtered, windowed, or compared.

example_21.py — run this from script.py

```
from datetime import datetime, timedelta

now = datetime.now()
print("Now:", now)

# Formatting a datetime into a partition-style path
partition_path = now.strftime("year=%Y/month=%m/day=%d")
print(partition_path)

# Parsing a timestamp string coming from a log file
log_line_ts = "2026-07-25 14:32:10"
parsed = datetime.strptime(log_line_ts, "%Y-%m-%d %H:%M:%S")
print("Parsed:", parsed, "| Hour:", parsed.hour)

# Date arithmetic - common for "last 7 days" style filters
seven_days_ago = now - timedelta(days=7)
print("7 days ago:", seven_days_ago.date())

event_times = [
    datetime(2026, 7, 20, 9, 0),
    datetime(2026, 7, 24, 15, 30),
    datetime(2026, 7, 10, 8, 0),
]
recent_events = [t for t in event_times if t >= seven_days_ago]
print("Recent events:", recent_events)
```

Practice task: Given a list of date strings in 'YYYY-MM-DD' format, parse them and print only the ones that fall on a weekend.

22 Regular Expressions for Data Cleaning

The `re` module matches and extracts text with patterns. `re.match`/`search` test for a pattern, `re.findall` extracts all matches, and `re.sub` replaces matched text.

WHY IT MATTERS FOR DATA ENGINEERING

Log parsing, extracting IDs embedded in file names, and validating formats (emails, phone numbers, IP addresses) in raw text data are everyday regex tasks that plain string methods cannot express well.

example_22.py — run this from script.py

```
import re

log_line = "2026-07-25 14:32:10 ERROR user=C-1042 failed to load order 0-98213"

# Extract a structured field with a capture group
match = re.search(r"user=(\S+)", log_line)
if match:
    print("User:", match.group(1))

# Extract all IDs matching a pattern
order_ids = re.findall(r"0-\d+", log_line)
print("Order IDs found:", order_ids)

# Validate a format (e.g. simple email check) before loading a row
def is_valid_email(value):
    pattern = r"^[a-zA-Z0-9_.-]+@[a-zA-Z0-9_.-]+\.[a-zA-Z]+$"
    return re.match(pattern, value) is not None

for email in ["a.smith@example.com", "not-an-email", "b@data.co"]:
    print(email, "->", is_valid_email(email))

# Cleaning: strip out anything that isn't a digit from a phone-like field
messy_phone = "+20 (100) 123-4567"
digits_only = re.sub(r"\D", "", messy_phone)
print(digits_only)
```

Practice task: Given a list of file names like `'sales_2026_07_25.csv'`, use regex to extract just the date part from each.

23 The collections Module

`collections` extends the built-in types: `Counter` tallies items in one line, `defaultdict` auto-initializes missing keys, and `namedtuple` gives a lightweight, readable record type.

WHY IT MATTERS FOR DATA ENGINEERING

These remove boilerplate from exactly the group-and-count patterns that show up constantly in ETL work — `Counter` alone replaces most manual word-count / frequency-table code.

example_23.py — run this from script.py

```
from collections import Counter, defaultdict, namedtuple

# Counter: instant frequency table
statuses = ["paid", "pending", "paid", "failed", "paid", "pending"]
status_counts = Counter(statuses)
print(status_counts)
print(status_counts.most_common(1))  # most frequent status

# defaultdict: no need to check "if key not in dict" before appending
orders_by_customer = defaultdict(list)
data = [("C-101", 50), ("C-102", 30), ("C-101", 20)]
for customer, amount in data:
    orders_by_customer[customer].append(amount)
print(dict(orders_by_customer))

# namedtuple: a lightweight, self-documenting record
Order = namedtuple("Order", ["order_id", "amount", "status"])
o = Order("O-1", 120.50, "paid")
print(o.order_id, o.amount, o.status)
```

Practice task: Use `Counter` to find the most common word in a sentence of your choice, ignoring case.

24 Working with Files & Paths (os, pathlib)

`pathlib.Path` is the modern, object-oriented way to build and inspect file paths, list directory contents, and check whether files exist — portable across operating systems, unlike hand-built path strings.

WHY IT MATTERS FOR DATA ENGINEERING

Discovering which files landed in an ingestion folder, filtering by extension, and building partitioned output paths are daily tasks when a pipeline reads from or writes to a filesystem or data lake.

example_24.py — run this from script.py

```
from pathlib import Path

data_dir = Path("data_dir")
data_dir.mkdir(exist_ok=True)

# Create a few sample files to work with
(data_dir / "sales_2026_07_23.csv").write_text("id,amount\n1,10\n")
(data_dir / "sales_2026_07_24.csv").write_text("id,amount\n2,20\n")
(data_dir / "readme.txt").write_text("notes")

# List only CSV files in the directory
csv_files = sorted(data_dir.glob("*.csv"))
print("CSV files found:", [f.name for f in csv_files])

for f in csv_files:
    print(f.name, "-> exists:", f.exists(), "| size (bytes):", f.stat().st_size)

# Building a partitioned output path cleanly, cross-platform
output_path = data_dir / "processed" / "year=2026" / "month=07" / "part-0001.csv"
output_path.parent.mkdir(parents=True, exist_ok=True)
print("Would write to:", output_path)
```

Practice task: List all '.txt' files in a folder and print their total combined size in bytes.

25 Capstone: A Mini ETL Pipeline

This final example wires nearly everything above — functions, exception handling, comprehensions, generators, CSV/JSON, and collections — into one small but complete Extract-Transform-Load pipeline.

WHY IT MATTERS FOR DATA ENGINEERING

This is the shape every real pipeline takes, whether it's five lines of Python or a Spark job spanning a thousand machines: pull raw records in, clean and validate them, aggregate, and write the result somewhere durable.

example_25.py — run this from script.py

```
import csv
from collections import Counter

RAW_ROWS = [
    "0-1,120.50,paid",
    "0-2,abc,pending",          # malformed amount
    "0-3,305.00,paid",
    "0-4,12.75,paid",
    "0-5,-9.00,failed",        # negative amount
]

def extract(raw_rows):
    """Yield raw string rows lazily (generator)."""
    for row in raw_rows:
        yield row

def transform(rows):
    """Parse, validate, and clean each row; skip bad ones."""
    clean_rows = []
    errors = []
    for row in rows:
        try:
            order_id, amount_str, status = row.split(",")
            amount = float(amount_str)
            if amount < 0:
                raise ValueError("negative amount")
            clean_rows.append({"order_id": order_id, "amount": amount, "status": status})
        except ValueError as e:
            errors.append((row, str(e)))
    return clean_rows, errors

def aggregate(clean_rows):
    """Reduce phase: totals per status."""
    totals = {}
    for row in clean_rows:
        totals[row["status"]] = totals.get(row["status"], 0) + row["amount"]
    return totals

def load(clean_rows, path="clean_orders.csv"):
    with open(path, "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=["order_id", "amount", "status"])
        writer.writeheader()
        writer.writerows(clean_rows)

# ---- Run the pipeline ----
```

```
raw = extract(RAW_ROWS)
clean, errors = transform(raw)
totals = aggregate(clean)
load(clean)

print("Clean rows:", clean)
print("Errors:", errors)
print("Totals by status:", {k: round(v, 2) for k, v in totals.items()})
print("Status counts:", Counter(r["status"] for r in clean))
print("Saved clean_orders.csv")
```

Practice task: Extend the capstone pipeline to also write a JSON summary file containing the totals-by-status dictionary.